

YAPPR

Security, Cryptography & Dataflow Deep Dive

A comprehensive technical breakdown of how Yappr protects user data with Zero-Knowledge End-to-End Encryption, manages cryptographic keys, secures authentication flows, and defends against common web threats.

ECDH P-256

AES-256-GCM

bcrypt

JWT

Zero-Knowledge

Web Crypto API

Version: 1.0 · **Date:** August 21, 2026 · **Classification:** Internal

Yappr — Security, Cryptography & Dataflow Deep Dive

*A complete technical breakdown of how **Yappr** protects user data, encrypts messages end-to-end, manages cryptographic keys, handles password resets safely, and defends against common web attacks — all explained in plain language with every relevant function and file location listed.*

Table of Contents

1. High-Level Security Architecture
2. Authentication & Session Management (JWT)
3. Password Hashing with bcrypt
4. End-to-End Encryption (E2EE) — The Core
 - 4.1 ECDH Key Agreement (Diffie-Hellman on Elliptic Curves)
 - 4.2 AES-256-GCM Symmetric Encryption
 - 4.3 The Full Message Encryption Dataflow
5. Key Lifecycle & Envelope Encryption
 - 5.1 Key Generation at Signup
 - 5.2 IndexedDB — Local Private Key Storage
 - 5.3 Password Envelope (Server Backup #1)
 - 5.4 6-Digit PIN Envelope (Server Backup #2 — Safety Net)
 - 5.5 Master Key Derivation Pipeline
6. Key Recovery & Restoration Flows
 - 6.1 Login Key Restoration
 - 6.2 PIN Recovery Flow
 - 6.3 Key Regeneration (Last Resort)
7. Password Reset (Forgot Password)
 - 7.1 How the Reset Token is Generated
 - 7.2 How the Reset Token is Verified
 - 7.3 Impact on E2EE Keys
8. Change Password Flow
9. PIN Brute-Force Protection
10. Server-Side Hardening & Middleware
11. Real-Time Transport Security (Socket.IO)
12. Message Storage & What the Server Sees
13. Group Messages — Current Security Model
14. Complete Function Reference
15. Dataflow Diagrams
16. Security Threat Model Summary

1. High-Level Security Architecture

Yappr implements a **Zero-Knowledge End-to-End Encryption (E2EE)** model for direct (1:1) messages. This means:

- The **server never sees plaintext messages** — it only stores opaque ciphertext blobs.
- The **server never holds private keys** — it only stores *encrypted envelopes* of the private key that only the user can decrypt with their password or PIN.
- **Encryption and decryption happen entirely on the client** (the browser) using the **Web Crypto API (SubtleCrypto)**.
- Even if the database is breached, the attacker cannot read message content or recover private keys without the user's password or PIN.

Architecture Diagram

```
graph TB
  subgraph Client ["Browser (Client-Side)"]
    A["Web Crypto API (SubtleCrypto)"]
    B["IndexedDB (Private Key Store)"]
    C["Zustand State (Auth/Chat Stores)"]
  end
  subgraph Server ["Node.js Backend"]
    D["Express + Helmet + Rate Limiter"]
    E["MongoDB (Mongoose)"]
    F["Socket.IO (Real-Time)"]
  end
  A -- "ECDH Key Agreement" --> A
  A -- "AES-GCM Encrypt/Decrypt" --> A
  B -- "Private Key JWK" --> A
  C -- "Ciphertext + IV" --> D
  D -- "Store encrypted blob" --> E
  D -- "Emit to recipient" --> F
  F -- "Ciphertext + IV" --> C
  C -- "Decrypt with shared key" --> A
```

2. Authentication & Session Management (JWT)

Yappr uses **JSON Web Tokens (JWT)** stored in **httpOnly cookies** for session management. This is a stateless auth strategy — the server doesn't maintain session state; instead, the signed JWT itself proves identity.

How It Works

1. On **signup** or **login**, the server creates a JWT containing the user's `userId`, signed with a secret (`JWT_SECRET` from `.env`).
2. This JWT is set as an **httpOnly, sameSite=strict, secure** cookie — meaning:
 - **httpOnly**: JavaScript running in the browser cannot read the cookie (protects against **XSS — Cross-Site Scripting**).
 - **sameSite: strict**: The cookie is not sent on cross-origin requests (protects against **CSRF — Cross-Site Request Forgery**).
 - **secure**: In production, the cookie is only sent over HTTPS (protects against **MitM — Man-in-the-Middle** attacks).
3. The JWT expires after **7 days**.

Route Protection

Every protected API route passes through the `protectRoute` middleware, which:

1. Reads the `jwt` cookie from the request.
2. Verifies it with `jwt.verify()` using `JWT_SECRET`.
3. Looks up the user in MongoDB (excluding the password hash).
4. Attaches the user object to `req.user` so controllers can use it.

! IMPORTANT

The password is ****never**** stored in the JWT or returned to the client. The ``select("-password")`` call explicitly excludes it from all query results attached to ``req.user``.

Functions & Locations

FUNCTION	FILE	PURPOSE
<code>generateToken(userId, res)</code>	<code>backend/src/lib/utils.js</code>	Creates a signed JWT with 7-day expiry, sets it as an httpOnly cookie
<code>protectRoute(req, res, next)</code>	<code>backend/src/middleware/auth.middleware.js</code>	Middleware that verifies the JWT cookie and loads the user onto <code>req.user</code>

3. Password Hashing with bcrypt

User passwords are **never stored in plaintext**. Yappr uses **bcrypt** — an adaptive, salted, one-way hashing algorithm specifically designed for passwords.

How It Works

1. **On signup**, a random **salt** is generated using `bcrypt.genSalt(10)` (the `10` is the **cost factor** — meaning $2^{10} = 1,024$ rounds of the Blowfish-based key schedule).
2. The password + salt are fed through bcrypt to produce a **60-character hash string** that encodes the algorithm version, cost factor, salt, and hash together.
3. This hash is stored in the `password` field of the `User` document in MongoDB.
4. **On login**, `bcrypt.compare(plainPassword, storedHash)` re-derives the hash from the entered password using the embedded salt and compares it to the stored hash.

i NOTE

bcrypt is intentionally ****slow**** — the cost factor makes brute-force attacks computationally expensive. At cost factor 10, each hash takes ~100ms, making mass guessing impractical.

Functions & Locations

FUNCTION	FILE	PURPOSE
<code>signup(req, res)</code>	<code>backend/src/controllers/auth.controller.js</code>	Hashes password with <code>bcrypt.genSalt(10)</code> + <code>bcrypt.hash()</code> before saving
<code>login(req, res)</code>	<code>backend/src/controllers/auth.controller.js</code>	Verifies entered password against stored hash with <code>bcrypt.compare()</code>
<code>changePassword(req, res)</code>	<code>backend/src/controllers/auth.controller.js</code>	Verifies old password, hashes new password, saves
<code>resetPassword(req, res)</code>	<code>backend/src/controllers/auth.controller.js</code>	Hashes the new password after token validation

4. End-to-End Encryption (E2EE) — The Core

This is the heart of Yappr's security. Every direct message between two users is encrypted so that **only the sender and the recipient** can read it. The server, the database, a compromised admin — none of them can decrypt the message content.

4.1 ECDH Key Agreement (Diffie-Hellman on Elliptic Curves)

ECDH stands for **Elliptic Curve Diffie-Hellman**. It's a key agreement protocol — a way for two parties to independently compute the same shared secret without ever transmitting that secret.

How it works in Yappr:

- Each user generates an **ECDH P-256 key pair** (a private key and a public key) on the **P-256 elliptic curve** (also called `secp256r1` or `prime256v1` — a NIST-standardized curve with 128-bit security level).
- The **public key** is uploaded to the server and stored in the user's profile (visible to all users).
- The **private key** stays on the user's device, stored in **IndexedDB** (browser-local storage).
- When Alice wants to message Bob, her browser takes:
 - Alice's private key** (from IndexedDB)
 - Bob's public key** (fetched from the server)
 - And computes a **shared secret** using the ECDH algorithm.
- Bob's browser does the same with **Bob's private key** + **Alice's public key**, and mathematically arrives at the **exact same shared secret**.
- This shared secret is then used as an **AES-256-GCM symmetric encryption key**.

TIP

The beauty of ECDH is that the shared secret is never transmitted. Even if someone intercepts both public keys and all network traffic, they cannot compute the shared secret without one of the private keys. This is the ****Elliptic Curve Discrete Logarithm Problem (ECDLP)**** — believed to be computationally infeasible.

Key Properties:

- **Curve:** P-256 (256-bit prime field, 128-bit security level)
- **Key format:** JWK (JSON Web Key) — a standard JSON representation of cryptographic keys
- **The shared key is non-extractable:** once derived, the AES key cannot be exported from the browser's crypto engine — it can only be used for encryption/decryption operations

Functions & Locations

FUNCTION	FILE	PURPOSE
<code>generateECDHKeyPair()</code>	<code>frontend/src/lib/crypto.js</code>	Generates an ECDH P-256 key pair using <code>crypto.subtle.generateKey()</code>
<code>exportKeyToJWK(key)</code>	<code>frontend/src/lib/crypto.js</code>	Exports a <code>CryptoKey</code> to JWK format using <code>crypto.subtle.exportKey()</code>
<code>importPublicKeyFromJWK(jwk)</code>	<code>frontend/src/lib/crypto.js</code>	Imports a public key from JWK for use in <code>deriveKey</code>
<code>importPrivateKeyFromJWK(jwk)</code>	<code>frontend/src/lib/crypto.js</code>	Imports a private key from JWK with <code>deriveKey</code> + <code>deriveBits</code> usage
<code>deriveSharedKey(myPrivateKey, recipientPublicKey)</code>	<code>frontend/src/lib/crypto.js</code>	Performs ECDH key agreement → produces an AES-GCM 256-bit shared key

4.2 AES-256-GCM Symmetric Encryption

Once the shared key is derived (via ECDH), actual message encryption uses **AES-256-GCM**:

- **AES** = Advanced Encryption Standard — the most widely used symmetric cipher.
- **256** = 256-bit key length — the strongest standard key size (2^{256} possible keys).
- **GCM** = Galois/Counter Mode — an **authenticated encryption** mode that provides both **confidentiality** (nobody can read the data) and **integrity** (nobody can tamper with the data without detection).

How encryption works for each message:

1. A fresh **12-byte IV (Initialization Vector)** is generated randomly using `crypto.getRandomValues()`. Every message gets a unique IV — this is critical because reusing an IV with the same key would break GCM's security guarantees.
2. The plaintext message is encoded to bytes using `TextEncoder`.
3. `crypto.subtle.encrypt()` with AES-GCM produces the **ciphertext** + an embedded **authentication tag** (16 bytes appended to the ciphertext by GCM).
4. Both the ciphertext and IV are **Base64-encoded** for safe transport over JSON/HTTP.

How decryption works:

1. The Base64-encoded ciphertext and IV are decoded back to `ArrayBuffer`.
2. `crypto.subtle.decrypt()` with AES-GCM verifies the authentication tag first (integrity check), then decrypts.
3. If tampered with, decryption will throw an error — the tag won't match.

4. On success, `TextDecoder` converts the raw bytes back to a readable string.

Functions & Locations

FUNCTION	FILE	PURPOSE
<code>encryptText(plainText, sharedKey)</code>	<code>frontend/src/lib/crypto.js</code>	Encrypts plaintext → <code>{ciphertextBase64, ivBase64}</code> using AES-256-GCM with a random 12-byte IV
<code>decryptText(ciphertextBase64, ivBase64, sharedKey)</code>	<code>frontend/src/lib/crypto.js</code>	Decrypts AES-256-GCM ciphertext back to plaintext string
<code>arrayBufferToBase64(buffer)</code>	<code>frontend/src/lib/crypto.js</code>	Converts ArrayBuffer to Base64 string for JSON transport
<code>base64ToArrayBuffer(base64)</code>	<code>frontend/src/lib/crypto.js</code>	Converts Base64 string back to ArrayBuffer for crypto operations

4.3 The Full Message Encryption Dataflow

Here's exactly what happens from the moment Alice types a message to Bob seeing it:

Sequence Diagram

Alice's Browser

Yappr Server

MongoDB

Bob's Browser

 Alice types "Hello Bob!"

Alice → Alice: getPrivateKeyLocally(aliceId) → Alice's Private Key JWK

Alice → Alice: importPrivateKeyFromJWK(alicePrivJwk) → CryptoKey

Alice → Alice: importPublicKeyFromJWK(bob.publicKey) → CryptoKey

Alice → Alice: deriveSharedKey(alicePriv, bobPub) → AES-GCM-256 shared key

Alice → Alice: encryptText("Hello Bob!", sharedKey) → {ciphertextBase64, ivBase64}

Alice → Server: POST /api/messages/send/:bobId {text: ciphertext, iv: iv, isEncrypted: true}

Server → DB: Save message document (ciphertext stored as-is)

Server → Bob: Socket.IO emit "newMessage" {text: ciphertext, iv: iv, isEncrypted: true}

Bob → Bob: getPrivateKeyLocally(bobId) → Bob's Private Key JWK

Bob → Bob: importPrivateKeyFromJWK(bobPrivJwk) → CryptoKey

Bob → Bob: importPublicKeyFromJWK(alice.publicKey) → CryptoKey

Bob → Bob: deriveSharedKey(bobPriv, alicePub) → SAME AES-GCM-256 shared key

Bob → Bob: decryptText(ciphertext, iv, sharedKey) → "Hello Bob!"

 Bob reads "Hello Bob!"

Key points:

- The server **only sees** the Base64 ciphertext blob and the IV — it cannot decrypt.
- Both Alice and Bob independently derive the **same shared key** from their own private key + the other's public key. This is the ECDH mathematical property.
- The shared key is **cached in memory** (`sharedKeysCache` Map in `useChatStore.js`) so ECDH derivation doesn't run on every single message.

Functions & Locations (Chat Store — Orchestration Layer)

FUNCTION	FILE	PURPOSE
<code>getSharedKeyForUser(otherUser)</code>	<code>frontend/src/store/useChatStore.js</code>	Derives (or cache-returns) the shared AES key for a specific user
<code>sendMessage(messageData)</code>	<code>frontend/src/store/useChatStore.js</code>	Encrypts message text before sending; keeps plaintext locally for display
<code>getMessages(userId)</code>	<code>frontend/src/store/useChatStore.js</code>	Fetches messages from server, decrypts all <code>isEncrypted</code> messages client-side
<code>subscribeToMessages()</code>	<code>frontend/src/store/useChatStore.js</code>	Socket.IO listener that decrypts incoming real-time messages
<code>forwardMessage(targetUserId, messageData)</code>	<code>frontend/src/store/useChatStore.js</code>	Re-encrypts forwarded messages with the target user's shared key

5. Key Lifecycle & Envelope Encryption

5.1 Key Generation at Signup

When a user signs up, the **frontend** (not the server!) generates a fresh ECDH P-256 key pair:

1. `generateECDHKeyPair()` → produces `{publicKey, privateKey}` as `CryptoKey` objects.
2. Both keys are exported to **JWK (JSON Web Key)** format using `exportKeyToJWK()`.
3. The **private key JWK** is saved to the browser's **IndexedDB** via `storePrivateKeyLocally()`.
4. The **public key JWK** is uploaded to the server via `PUT /api/auth/e2ee-keys`.
5. The private key is also **envelope-encrypted** with the user's password (and optionally a 6-digit PIN) and the encrypted blobs are uploaded to the server as backup.

Functions & Locations

FUNCTION	FILE	PURPOSE
<code>signup(data)</code>	<code>frontend/src/store/useAuthStore.js</code>	Full signup flow: keygen → local storage → server upload with envelopes
<code>updateE2EEKeys(req, res)</code>	<code>backend/src/controllers/auth.controller.js</code>	Server endpoint that saves public key and encrypted envelopes to user document

5.2 IndexedDB — Local Private Key Storage

The user's **raw private key** (in JWK format) is stored in the browser's **IndexedDB** — a client-side database that persists across sessions.

- **Database name:** `Yappr_E2EE_Store`
- **Object store name:** `private_keys`
- **Key:** User's MongoDB `_id`
- **Value:** The private key JWK object

⚠ WARNING

IndexedDB is local to the browser. If the user clears browser data, switches browsers, or uses a different device, the private key is gone from IndexedDB. That's why server-side encrypted backups (password envelope + PIN envelope) exist.

Functions & Locations

FUNCTION	FILE	PURPOSE
<code>openDB()</code>	<code>frontend/src/lib/crypto.js</code>	Opens (or creates) the IndexedDB database
<code>storePrivateKeyLocally(userId, privateKeyJwk)</code>	<code>frontend/src/lib/crypto.js</code>	Stores the private key JWK in IndexedDB keyed by <code>userId</code>
<code>getPrivateKeyLocally(userId)</code>	<code>frontend/src/lib/crypto.js</code>	Retrieves the private key JWK from IndexedDB (returns <code>null</code> if absent)

5.3 Password Envelope (Server Backup #1)

The "password envelope" is the user's private key **encrypted with a key derived from their account password**. This encrypted blob is stored on the server. It means:

- If the user logs in from a new device (where IndexedDB is empty), the server sends back the encrypted blob.
- The frontend uses the entered password to derive the same master key and decrypt the private key.
- The server **cannot** decrypt this blob — it doesn't know the plaintext password (only the bcrypt hash).

What's stored on the server (User document):

FIELD	CONTENT
<code>encryptedPrivateKey</code>	Base64-encoded AES-256-GCM ciphertext of the private key JWK
<code>keySalt</code>	The bcrypt salt used in the master key derivation pipeline
<code>keyIv</code>	The 12-byte IV used for the AES-GCM encryption (Base64)

5.4 6-Digit PIN Envelope (Server Backup #2 — Safety Net)

The PIN envelope is an **independent second backup** of the same private key, encrypted with a key derived from a user-chosen 6-digit PIN. It exists as a **recovery safety net** for when:

- The user resets their password (which invalidates the password envelope).
- The user's IndexedDB is cleared and they forgot their password.

What's stored on the server (User document):

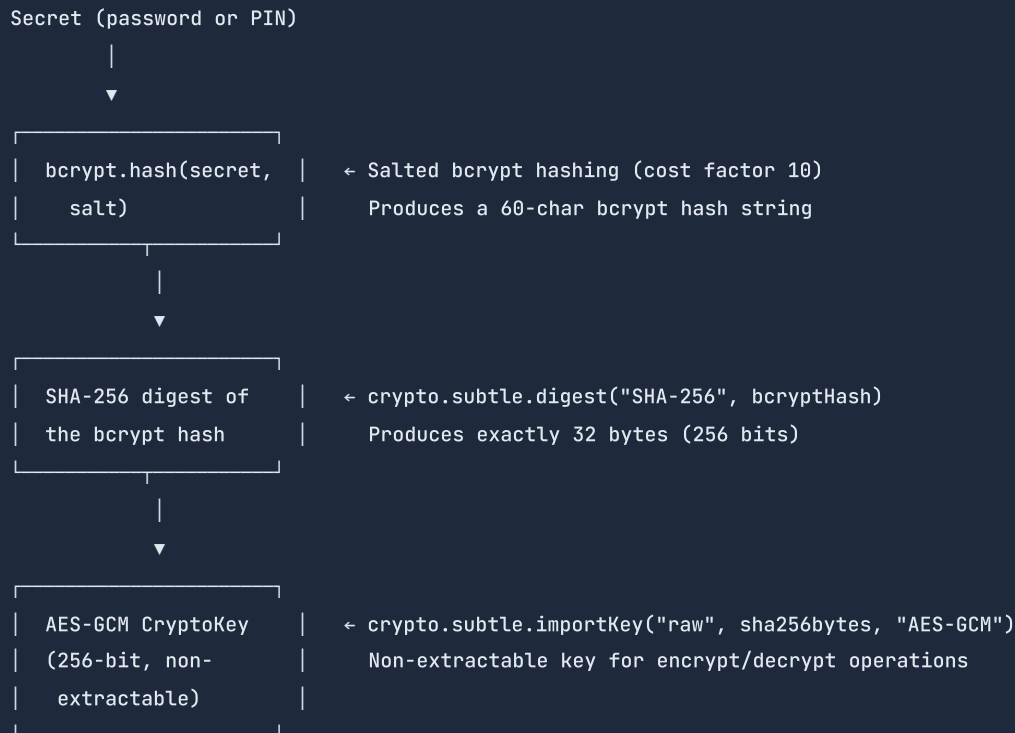
FIELD	CONTENT
<code>pinEncryptedPrivateKey</code>	Base64-encoded AES-256-GCM ciphertext of the private key JWK
<code>pinSalt</code>	The bcrypt salt used in the PIN-based master key derivation
<code>pinIv</code>	The 12-byte IV for the PIN-based AES-GCM encryption (Base64)

! IMPORTANT

The password envelope and PIN envelope encrypt the **same private key** but with **different derived master keys** and **different IVs**. They are completely independent — knowing one doesn't help you crack the other.

5.5 Master Key Derivation Pipeline

Both the password envelope and PIN envelope use the same derivation pipeline to turn a user's secret (password or PIN) into an AES-256-GCM encryption key:



Why this pipeline?

1. **bcrypt** provides salt + computational cost (slows down brute-force).
2. **SHA-256** condenses the 60-byte bcrypt output into exactly 32 bytes (256 bits) — the exact size required for an AES-256 key.
3. The resulting `CryptoKey` is marked **non-extractable** — once imported, it can only be used for encrypt/decrypt operations, not exported.

Functions & Locations

FUNCTION	FILE	PURPOSE
<code>deriveMasterKeyFromSecret(secret, salt)</code>	<code>frontend/src/lib/crypto.js</code>	Full pipeline: secret → bcrypt → SHA-256 → AES-GCM CryptoKey
<code>encryptPrivateKeyWithSecret(privateKeyJwk, secret, existingSalt)</code>	<code>frontend/src/lib/crypto.js</code>	Derives master key, then encrypts the private key JWK → <code>{ciphertextBase64, salt, ivBase64}</code>
<code>decryptPrivateKeyWithSecret(ciphertextBase64, salt, ivBase64, secret)</code>	<code>frontend/src/lib/crypto.js</code>	Derives master key, then decrypts ciphertext → private key JWK

6. Key Recovery & Restoration Flows

6.1 Login Key Restoration

When a user logs in, the `initializeE2EEKeys()` function runs a **priority-ordered recovery cascade**:

1. Check IndexedDB for local private key
 - └ Found? → Done! (also repairs any missing server backups if credentials available)
 - └ Not found? → Continue...
2. Try decrypting the PASSWORD ENVELOPE from the server
 - └ Success? → Store recovered key in IndexedDB → Done!
 - └ Failed/unavailable? → Continue...
3. Check if a PIN ENVELOPE exists on the server
 - └ Exists? → Show PinRecoveryModal (user enters their 6-digit PIN)
 - └ Doesn't exist? → Continue...
4. No keys anywhere → Generate BRAND NEW key pair (first-time setup / legacy migration)

Functions & Locations

FUNCTION	FILE	PURPOSE
<code>initializeE2EEKeys(user, password, pin)</code>	<code>frontend/src/store/useAuthStore.js</code>	The master key recovery cascade — runs on login and auth check

6.2 PIN Recovery Flow

When the PIN recovery modal appears (because IndexedDB is empty and the password envelope failed/is unavailable):

1. User enters their 6-digit PIN.
2. `recoverKeyWithPin(pin)` calls `decryptPrivateKeyWithSecret()` with the PIN envelope data from the server.
3. On success, the recovered private key JWK is stored in IndexedDB via `storePrivateKeyLocally()`.
4. If the user also provides their current password, a fresh password envelope is re-created and uploaded to the server.
5. A successful PIN attempt resets the `pinFailedAttempts` counter on the server via `POST /api/auth/pin-attempt`.

Functions & Locations

FUNCTION	FILE	PURPOSE
<code>recoverKeyWithPin(pin, currentPassword)</code>	<code>frontend/src/store/useAuthStore.js</code>	Decrypts PIN envelope → stores locally → optionally restores password envelope
<code>recordPinAttempt(req, res)</code>	<code>backend/src/controllers/auth.controller.js</code>	Server-side tracking of PIN attempt success/failure for brute-force protection
<code>PinRecoveryModal</code>	<code>frontend/src/components/PinRecoveryModal.jsx</code>	UI component for PIN entry and key recovery

6.3 Key Regeneration (Last Resort)

If the user has **lost both their password and PIN**, they can choose to **generate entirely new keys**. This is a destructive action:

- A brand-new ECDH key pair is created.
- New encrypted backups are uploaded.
- **All past encrypted messages become permanently unreadable** because the old private key is lost.
- New conversations going forward will work fine.

The "Forgot your 6-digit PIN?" link in the `PinRecoveryModal` leads to a confirmation screen that warns the user about this trade-off before proceeding.

Functions & Locations

FUNCTION	FILE	PURPOSE
<code>handleRegenerateKeys()</code>	<code>frontend/src/components/PinRecoveryModal.jsx</code>	Triggers key regeneration via <code>initializeE2EEKeys()</code>

7. Password Reset (Forgot Password)

7.1 How the Reset Token is Generated

1. User submits their email to `POST /api/auth/forgot-password`.
2. Server generates **20 random bytes** using Node.js `crypto.randomBytes(20)` → converts to hex string. This is the **raw reset token**.
3. The raw token is **SHA-256 hashed** using `crypto.createHash("sha256")` before being stored in the database. This means:

- The database stores the **hash** of the token, not the token itself.
 - Even if the database is breached, the attacker can't use the stored hash to construct a valid reset URL.
4. A `resetPasswordExpire` timestamp is set to **15 minutes** from now.
 5. The **raw (unhashed) token** is embedded in a reset URL and emailed to the user via **Nodemailer** (using Gmail SMTP).

Raw Token (in email link): `a3f8c1e2...` ← This is what the user clicks
 Stored in DB: `sha256(a3f8c1e2...) = 7b2d9f...` ← Hash of the token

7.2 How the Reset Token is Verified

1. User clicks the reset link → `PUT /api/auth/reset-password/:token`.
2. Server takes the raw token from the URL, **SHA-256 hashes it**, and looks for a matching `resetPasswordToken` in the database WHERE `resetPasswordExpire > Date.now()`.
3. If found and not expired → the new password is bcrypt-hashed and saved.
4. Both `resetPasswordToken` and `resetPasswordExpire` are cleared.

7.3 Impact on E2EE Keys

● CAUTION

When a password reset occurs, the ****password envelope is invalidated**** (set to `null`) because the old password that encrypted it is no longer known. However, the ****PIN envelope is preserved****. This is intentional — the PIN is the safety net for exactly this scenario.

After resetting the password and logging in:

1. `initializeE2EEKeys()` runs its recovery cascade.
2. The password envelope is `null` → cannot decrypt from password.
3. The PIN envelope exists → the `PinRecoveryModal` opens.
4. User enters PIN → private key is recovered → new password envelope is generated with the new password.

If the user never set a PIN: They're out of luck for past messages. `initializeE2EEKeys()` falls through to step 4 and generates a brand-new key pair.

Functions & Locations

FUNCTION	FILE	PURPOSE
<code>forgotPassword(req, res)</code>	<code>backend/src/controllers/auth.controller.js</code>	Generates token, hashes it, stores hash, emails raw token
<code>resetPassword(req, res)</code>	<code>backend/src/controllers/auth.controller.js</code>	Verifies token hash, resets password, invalidates password envelope
<code>forgotPassword(email)</code>	<code>frontend/src/store/useAuthStore.js</code>	Frontend action that calls the forgot-password API
<code>resetPassword(token, password)</code>	<code>frontend/src/store/useAuthStore.js</code>	Frontend action that submits the new password with the token

8. Change Password Flow

Unlike "Forgot Password" (which is destructive to the password envelope), **Change Password** is a smooth, non-destructive operation because the user knows their current password:

1. User enters current password + new password on the `ProfilePage`.
2. Frontend verifies the current password via the backend (`bcrypt.compare`).
3. Frontend retrieves the private key from IndexedDB (or decrypts it from the current password envelope).
4. Frontend **re-encrypts the same private key** with the **new password** using `encryptPrivateKeyWithSecret()`.
5. Both the new bcrypt-hashed password AND the new password envelope are sent to the server **atomically** in a single `PUT /api/auth/change-password` request.
6. The server updates everything at once — password hash, `encryptedPrivateKey`, `keySalt`, `keyIv`.

TIP

This atomic approach ensures the password and its corresponding envelope are always in sync. There's no window where the password is updated but the envelope still uses the old password.

Functions & Locations

FUNCTION	FILE	PURPOSE
<code>changePassword(currentPassword, newPassword)</code>	<code>frontend/src/store/useAuthStore.js</code>	Frontend: retrieves key → re-encrypts with new password → sends to server
<code>changePassword(req, res)</code>	<code>backend/src/controllers/auth.controller.js</code>	Backend: verifies old password → hashes new → saves new envelope atomically

9. PIN Brute-Force Protection

Since a 6-digit PIN has only 1,000,000 combinations, brute-force is a concern. Yappr implements **server-side rate limiting** for PIN attempts:

FIELD	TYPE	PURPOSE
<code>pinFailedAttempts</code>	<code>Number</code>	Counter of consecutive failed PIN attempts
<code>pinLockedUntil</code>	<code>Date</code>	Timestamp until which PIN entry is locked

Rules:

- After **5 consecutive failed PIN attempts**, the account is **locked for 15 minutes** (`pinLockedUntil = Date.now() + 15 * 60 * 1000`).
- A **successful** PIN attempt resets the counter to 0.
- While locked, the server returns `HTTP 429 Too Many Requests` with the remaining lockout time.

i NOTE

The bcrypt cost factor in the master key derivation pipeline also adds implicit rate-limiting — each PIN attempt takes ~100ms of CPU time on the client, making rapid automated guessing harder even before the server-side lockout kicks in.

Functions & Locations

FUNCTION	FILE	PURPOSE
<code>recordPinAttempt(req, res)</code>	<code>backend/src/controllers/auth.controller.js</code>	Increments failure counter, triggers 15-min lockout after 5 failures

Data Model Fields

Defined in `user.model.js` :

- `pinFailedAttempts` (line 66-68)
- `pinLockedUntil` (line 70-72)

10. Server-Side Hardening & Middleware

The Express server applies several security layers before any route handler runs:

10.1 Helmet (Secure HTTP Headers)

`helmet()` sets various HTTP headers that protect against common attacks:

- `X-Content-Type-Options: nosniff` — prevents MIME type sniffing
- `X-Frame-Options: DENY` — prevents clickjacking via iframes
- `Strict-Transport-Security` — enforces HTTPS
- `X-XSS-Protection` — enables browser XSS filter
- And many more...

10.2 MongoDB Sanitization (NoSQL Injection Prevention)

`mongoSanitize()` strips out any `$` or `.` characters from `req.body`, `req.query`, and `req.params`. This prevents **NoSQL injection attacks** like:

```
{ "email": { "$gt": "" } } // ← Would match ALL users without sanitization
```

10.3 CORS (Cross-Origin Resource Sharing)

CORS config restricts which origins can make API requests:

- **Development:** Only `http://localhost:5173` (Vite dev server)
- **Production:** Only the configured `CLIENT_URL`
- `credentials: true` — allows cookies to be sent cross-origin (required for JWT cookies)

10.4 Rate Limiting (Brute-Force Protection)

The `authLimiter` middleware applies to:

- `POST /api/auth/login`
- `POST /api/auth/signup`
- `POST /api/auth/forgot-password`

Rules: Maximum **20 requests per 15-minute window per IP address**. After exceeding, the server returns `HTTP 429 Too Many Requests`.

10.5 Request Size Limits

Body parser limits are set to **10MB** for both JSON and URL-encoded bodies — this accommodates Base64-encoded image uploads while preventing excessively large payloads.

11. Real-Time Transport Security (Socket.IO)

Socket.IO connections are **authenticated via JWT** using a custom middleware:

1. When a client connects, the server's **Socket.IO authentication middleware** runs.
2. It parses the `cookie` header from the WebSocket handshake request.
3. Extracts the `jwt` cookie and verifies it with `jwt.verify()`.
4. If valid, the `userId` is attached to the socket object (`socket.userId`).
5. If invalid or missing, the connection is **rejected** with an authentication error.

This means **unauthenticated users cannot connect to Socket.IO** — they won't receive real-time messages or online status updates.

CORS for Socket.IO

The **Socket.IO CORS config** mirrors the Express CORS config — same origin restrictions, same credential requirements.

Functions & Locations

FUNCTION	FILE	PURPOSE
<code>io.use((socket, next) => {...})</code>	<code>backend/src/lib/socket.js</code>	Socket.IO JWT authentication middleware
<code>getReceiverSocketId(userId)</code>	<code>backend/src/lib/socket.js</code>	Maps <code>userId</code> to their current socket ID for targeted message delivery
<code>connectSocket()</code>	<code>frontend/src/store/useAuthStore.js</code>	Client-side Socket.IO connection with <code>withCredentials: true</code>

12. Message Storage & What the Server Sees

The `Message` model stores:

FIELD	WHAT THE SERVER STORES
<code>senderId</code>	MongoDB ObjectId of the sender
<code>receiverId</code>	MongoDB ObjectId of the receiver (null for group messages)
<code>groupId</code>	MongoDB ObjectId of the group (null for DMs)
<code>text</code>	For encrypted DMs: Base64-encoded AES-256-GCM ciphertext (completely opaque). For unencrypted/group msgs: plaintext
<code>image</code>	Cloudinary URL (images are NOT E2EE currently)
<code>iv</code>	The Base64-encoded 12-byte IV used for this message's encryption
<code>isEncrypted</code>	Boolean flag indicating whether <code>text</code> is ciphertext
<code>isDeleted</code>	Soft-delete flag
<code>isSystemMessage</code>	System notification flag

⚠ WARNING

****Images are NOT end-to-end encrypted**** — they are uploaded to Cloudinary and stored as URLs. Only text messages are E2EE. This is a known limitation.

Functions & Locations

FUNCTION	FILE	PURPOSE
<code>sendMessage(req, res)</code>	<code>backend/src/controllers/message.controller.js</code>	Server handler: saves the encrypted payload as-is, emits via Socket.IO
<code>getMessages(req, res)</code>	<code>backend/src/controllers/message.controller.js</code>	Server handler: returns messages (still encrypted) for client-side decryption
<code>deleteMessage(req, res)</code>	<code>backend/src/controllers/message.controller.js</code>	Soft-deletes: replaces text with "This message was deleted", clears image

13. Group Messages — Current Security Model

i NOTE

****Group messages are NOT end-to-end encrypted**** in the current implementation. They are sent and stored as plaintext.

Group message security relies on:

1. **Membership verification:** The server checks that the sender is a member of the group before allowing the message (`sendGroupMessage`).
 2. **Timeout enforcement:** Timed-out users cannot send messages.
 3. **Admin-only messaging:** Groups can restrict sending to admins only via `permissions.allowMembersToSendMessages` .
 4. **Read authorization:** Only group members can fetch group messages (`getGroupMessages`).
-

14. Complete Function Reference

Frontend — Crypto Module (`frontend/src/lib/crypto.js`)

#	FUNCTION	LINES	DESCRIPTION
1	<code>arrayBufferToBase64(buffer)</code>	9-17	ArrayBuffer → Base64 string
2	<code>base64ToArrayBuffer(base64)</code>	19-27	Base64 string → ArrayBuffer
3	<code>openDB()</code>	30-42	Opens IndexedDB <code>Yappr_E2EE_Store</code> database
4	<code>storePrivateKeyLocally(userId, privateKeyJwk)</code>	44-53	Saves private key JWK to IndexedDB
5	<code>getPrivateKeyLocally(userId)</code>	55-64	Retrieves private key JWK from IndexedDB
6	<code>generateECDHKeyPair()</code>	67-76	Generates ECDH P-256 key pair
7	<code>exportKeyToJWK(key)</code>	79-81	Exports CryptoKey to JWK format
8	<code>importPublicKeyFromJWK(jwk)</code>	84-95	Imports ECDH public key from JWK
9	<code>importPrivateKeyFromJWK(jwk)</code>	98-109	Imports ECDH private key from JWK
10	<code>deriveSharedKey(myPrivateKey, recipientPublicKey)</code>	112-126	ECDH key agreement → AES-GCM-256 shared key
11	<code>encryptText(plainText, sharedKey)</code>	129-147	AES-256-GCM encryption with random 12-byte IV
12	<code>decryptText(ciphertextBase64, ivBase64, sharedKey)</code>	150-170	AES-256-GCM decryption with integrity verification
13	<code>deriveMasterKeyFromSecret(secret, salt)</code>	180-197	Secret → bcrypt → SHA-256 → AES-GCM key
14	<code>encryptPrivateKeyWithSecret(privateKeyJwk, secret, existingSalt)</code>	202-221	Envelope-encrypts private key with a secret
15	<code>decryptPrivateKeyWithSecret(ciphertextBase64, salt, ivBase64, secret)</code>	226-248	Envelope-decrypts private key ciphertext

Frontend — Auth Store (`frontend/src/store/useAuthStore.js`)

FUNCTION	LINES	DESCRIPTION
<code>initializeE2EEKeys(user, password, pin)</code>	35-131	Master key recovery cascade (4 priority levels)
<code>recoverKeyWithPin(pin, currentPassword)</code>	133-183	PIN-based private key recovery
<code>setupPinBackup(pin)</code>	185-212	Creates a PIN-encrypted envelope of the private key
<code>signup(data)</code>	228-283	Full signup: keygen → store → upload envelopes
<code>login(data)</code>	285-298	Login → connect socket → initialize E2EE keys
<code>changePassword(currentPassword, newPassword)</code>	351-396	Re-encrypts private key with new password atomically
<code>forgotPassword(email)</code>	325-334	Requests password reset email
<code>resetPassword(token, password)</code>	337-348	Submits new password with reset token
<code>connectSocket()</code>	398-413	Establishes authenticated Socket.IO connection

Frontend — Chat Store (`frontend/src/store/useChatStore.js`)

FUNCTION	LINES	DESCRIPTION
<code>getSharedKeyForUser(otherUser)</code>	18-43	Derives and caches ECDH shared key per user
<code>sendMessage(messageData)</code>	124-158	Encrypts → sends → stores plaintext locally
<code>getMessages(userId)</code>	94-122	Fetches + batch-decrypts all encrypted messages
<code>forwardMessage(targetUserId, messageData)</code>	160-189	Re-encrypts forwarded message for target user
<code>subscribeToMessages()</code>	206-248	Real-time decryption of incoming Socket.IO messages

Backend — Auth Controller (`backend/src/controllers/auth.controller.js`)

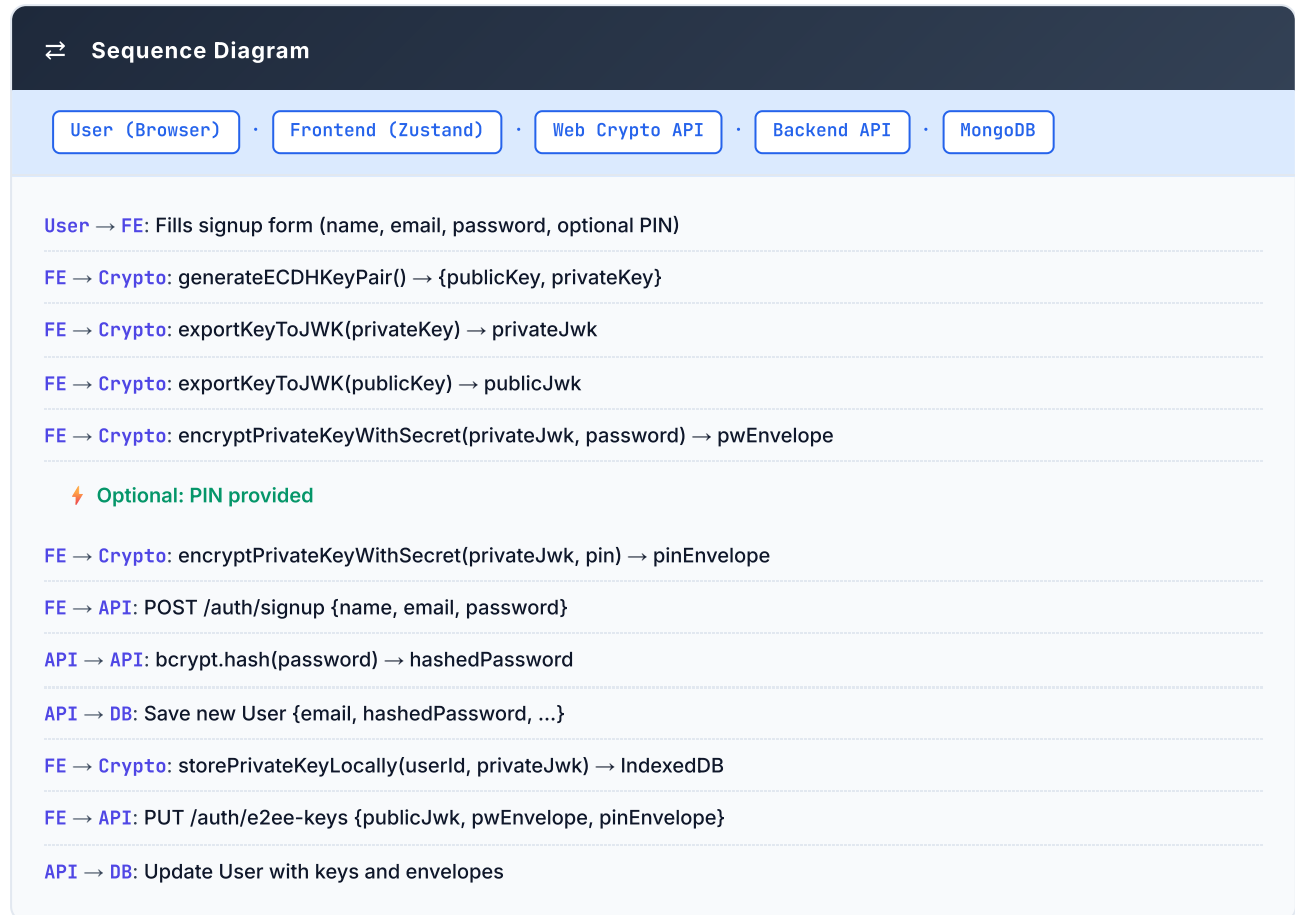
FUNCTION	LINES	DESCRIPTION
<code>signup(req, res)</code>	8-87	User registration with bcrypt password hashing
<code>login(req, res)</code>	89-131	Password verification + JWT issuance
<code>logout(req, res)</code>	133-141	Clears JWT cookie
<code>forgotPassword(req, res)</code>	172-269	Generates SHA-256-hashed reset token, sends email
<code>resetPassword(req, res)</code>	271-308	Verifies token, resets password, nullifies password envelope
<code>updateE2EEKeys(req, res)</code>	310-343	Saves/updates public key and encrypted envelopes
<code>changePassword(req, res)</code>	347-413	Verifies old password, saves new hash + new envelope atomically
<code>recordPinAttempt(req, res)</code>	415-447	PIN brute-force tracking with 15-min lockout

Backend — Middleware & Utils

FUNCTION	FILE	LINES	DESCRIPTION
<code>protectRoute(req, res, next)</code>	<code>auth.middleware.js</code>	4-32	JWT cookie verification middleware
<code>generateToken(userId, res)</code>	<code>utils.js</code>	3-16	Creates JWT + sets secure httpOnly cookie

15. Dataflow Diagrams

Signup + Key Generation Flow



Password Reset + Key Recovery Flow

Sequence Diagram

User (Browser)

Frontend

Backend API

MongoDB

Email Service

User → FE: Clicks "Forgot Password" → enters email

FE → API: POST /auth/forgot-password {email}

API → API: crypto.randomBytes(20) → rawToken

API → API: sha256(rawToken) → hashedToken

API → DB: Store hashedToken + expiry (15 min)

API → Email: Send email with resetUrl containing rawToken

 User clicks email link

User → FE: Enters new password on reset page

FE → API: PUT /auth/reset-password/:rawToken {password}

API → API: sha256(rawToken) → compare with stored hash

API → API: bcrypt.hash(newPassword) → newHash

API → DB: Save newHash, null out password envelope, keep PIN envelope

 User logs in with new password

FE → FE: initializeE2EEKeys() cascade

FE → FE: IndexedDB? No. Password envelope? Null. PIN envelope? Yes!

FE → User: Show PinRecoveryModal

User → FE: Enters 6-digit PIN

FE → FE: decryptPrivateKeyWithSecret(pinEnvelope, pin) → privateKeyJwk

FE → FE: storePrivateKeyLocally(userId, privateKeyJwk) → IndexedDB

FE → FE: encryptPrivateKeyWithSecret(privateKeyJwk, newPassword) → newPwEnvelope

FE → API: PUT /auth/e2ee-keys {newPwEnvelope}

 All past messages unlocked!

16. Security Threat Model Summary

THREAT	PROTECTION	HOW
Password brute-force	bcrypt (cost 10) + rate limiting (20/15min)	Slow hashing + IP-based request throttling
Session hijacking (XSS)	httpOnly + sameSite cookies	JS cannot access JWT; cookie not sent cross-origin
CSRF	sameSite=strict cookies	Browser won't include cookie in cross-origin requests
NoSQL injection	express-mongo-sanitize	Strips <code>\$</code> and <code>.</code> from request inputs
MitM (eavesdropping)	HTTPS (secure cookies) + E2EE	Cookie only sent over HTTPS; messages encrypted regardless of transport
Server-side data breach	Zero-Knowledge E2EE	Message text is ciphertext; private keys are encrypted with user-only secrets
Private key loss (device change)	Password + PIN envelopes	Server stores encrypted backups only the user can decrypt
Password reset key loss	PIN envelope preserved on reset	PIN envelope survives password changes — it's the recovery safety net
PIN brute-force	5-attempt lockout + bcrypt derivation cost	Server-side lockout (15 min) + ~100ms per attempt client-side
Clickjacking	Helmet (X-Frame-Options)	Browser blocks rendering in iframes
MIME sniffing attacks	Helmet (X-Content-Type-Options)	Browser respects declared content types
Unauthenticated Socket.IO access	JWT-authenticated WebSocket middleware	Connection rejected without valid JWT cookie
Unauthorized group message access	Server-side membership checks	Group controller verifies sender/reader is a group member

Data Model — User Security Fields

All fields from `user.model.js`:

FIELD	TYPE	DESCRIPTION
<code>password</code>	String	bcrypt-hashed password (never returned to client)
<code>publicKey</code>	Object	ECDH P-256 public key in JWK format (shared with other users)
<code>encryptedPrivateKey</code>	String	AES-256-GCM ciphertext of private key (encrypted with password-derived master key)
<code>keySalt</code>	String	bcrypt salt used in the password → master key derivation
<code>keyIv</code>	String	12-byte IV for the password envelope AES-GCM encryption (Base64)
<code>pinEncryptedPrivateKey</code>	String	AES-256-GCM ciphertext of private key (encrypted with PIN-derived master key)
<code>pinSalt</code>	String	bcrypt salt used in the PIN → master key derivation
<code>pinIv</code>	String	12-byte IV for the PIN envelope AES-GCM encryption (Base64)
<code>pinFailedAttempts</code>	Number	Consecutive failed PIN attempts (resets on success)
<code>pinLockedUntil</code>	Date	Lockout expiry timestamp (null if not locked)
<code>resetPasswordToken</code>	String	SHA-256 hash of the password reset token
<code>resetPasswordExpire</code>	Date	Expiry time for the password reset token (15 minutes)

Bottom line: Yappr's security is layered — transport security (HTTPS, httpOnly cookies), application security (helmet, rate limiting, mongo sanitization), authentication (JWT, bcrypt), and end-to-end encryption (ECDH + AES-256-GCM) combine to create a system where even the server operator cannot read user messages.